



Segurança em Banco de Dados e SQL Injection

Professor: Moisés Sousa

Objetivos da Aula

- Entender SQL Injection
- Aprender *Prepared Statements*
- Compreender hash de senha
- Conhecer boas práticas de segurança

Introdução

- Conectar ao banco é fácil.
- Conectar com segurança é o desafio real.

Código Vulnerável

```
$sql = "  
SELECT * FROM usuarios  
WHERE email = '$email'  
AND senha = '$senha'  
";
```

Problema da Concatenação

A entrada do usuário está sendo inserida diretamente dentro do SQL.

O que é SQL Injection?

- Técnica onde comandos SQL são injetados através das entradas da aplicação.

Exemplo de Ataque

Email: ' OR '1'='1

Senha: ' OR '1'='1

SQL Gerado pelo Ataque

```
SELECT * FROM usuarios  
WHERE email = " OR '1'='1'  
AND senha = " OR '1'='1'
```

Por que Funciona?

- `'1'='1'` sempre retorna verdadeiro.

Riscos do SQL Injection

- Bypass de login
- Roubo de dados
- Exclusão de tabelas

DROP TABLE

- `'; DROP TABLE usuarios; --`

Prepared Statements

```
$stmt = $conn->prepare("
SELECT * FROM usuarios
WHERE email = ?
AND senha = ?
");
```

Bind Param

```
$stmt->bind_param("ss", $email, $senha);
```

Execução Segura

- `$stmt->execute();`

Por que é Seguro?

- O banco separa comandos SQL dos dados enviados pelo usuário.

Senhas em Texto Puro

- Nunca armazene senhas diretamente no banco.

Hash de Senha

```
$senhaHash = password_hash(  
    $senha,  
    PASSWORD_DEFAULT  
);
```

Verificação de Senha

- `password_verify($senha, $senhaHash);`

Princípio do Menor Privilegio

- O sistema deve possuir apenas as permissões necessárias.

Usuário Limitado

- GRANT SELECT, INSERT, UPDATE
- ON curso.*
- TO 'sistema';

Resumo

- SQL Injection -> Prepared Statements
- Senha exposta -> Hash
- Root -> Usuário limitado

Teste Rápido - 02

Corrigir um sistema vulnerável usando Prepared Statements.